

VANJINATHAN S

Register Number: 192425231

Q38. Assembly Line Scheduling vs Word Break (DP Comparison)

Assembly Line Scheduling vs Word Break - A Dynamic Programming Comparison

Both Assembly Line Scheduling and Word Break look like very different problems on the surface - one is about choosing the fastest path through two factory assembly lines, and the other is about breaking a string into valid dictionary words. But both of them are solved efficiently using Dynamic Programming (DP) because they share the same underlying idea: the overall best answer can be built from the best answers of smaller sub-problems, and many of these sub-problems repeat. Below both problems are solved step by step, and then compared.

1. Assembly Line Scheduling - Problem and DP Solution

In this problem, a car chassis moves through two assembly lines, each with n stations. At each station i on a line, some fixed processing time is spent, and after each station the chassis can either continue on the same line or move to the other line, which costs a transfer time. There is also an entry time to start on either line and an exit time to leave either line at the end. The goal is to find the minimum total time to get the chassis from entry to exit.

Let the two lines be line 1 and line 2, with:

$S[i][j]$ = processing time at station j of line i ($i = 1, 2$)

$T[i][j]$ = transfer time from station j of line i to the other line

$e[i]$ = entry time into line i

$x[i]$ = exit time from line i

Define $f[i][j]$ as the minimum time to reach station j on line i . The recurrence relation is:

$f[1][1] = e[1] + S[1][1]$

$f[2][1] = e[2] + S[2][1]$

$f[1][j] = \min(f[1][j-1] + S[1][j], f[2][j-1] + T[2][j-1] + S[1][j])$ for $j > 1$

$f[2][j] = \min(f[2][j-1] + S[2][j], f[1][j-1] + T[1][j-1] + S[2][j])$ for $j > 1$

Finally, the answer is the minimum of both lines after adding the exit time:

$\text{Answer} = \min(f[1][n] + x[1], f[2][n] + x[2])$

At every station, the decision is simple: stay on the same line, or pay the transfer cost and switch from the other line - whichever gives the smaller total time so far. Since $f[i][j]$ only depends on $f[1][j-1]$ and $f[2][j-1]$, the table can be filled from $j = 1$ to n in a single pass, either using a $2 \times n$ table (bottom-up tabulation) or with just two variables if only the final answer is needed.

2. Word Break - Problem and DP Solution

In this problem, a string s and a dictionary of words (a set) are given. The task is to determine whether s can be split into a sequence of one or more dictionary words, one after another with no leftover characters. For example, "icecream" can be split into "ice" and "cream" if both are in the dictionary.

Let $dp[i]$ be true if the prefix $s[0..i-1]$ (the first i characters) can be fully segmented into dictionary words. The recurrence relation is:

$dp[0] = \text{true}$ (empty prefix needs no words)

$dp[i] = \text{OR over all } j \text{ from } 0 \text{ to } i-1 \text{ of: } (dp[j] \text{ AND } s[j..i-1] \text{ is in dictionary})$

The final answer is $dp[n]$, where n is the length of the string.

Here the decision at each position i is: try every possible "last word" ending at position i , that is, every split point j , and check if the part before it ($dp[j]$) was already segmentable and the remaining part ($s[j..i-1]$) is a valid dictionary word. If even one such split works, $dp[i]$ becomes true. This is filled from $i = 1$ to n , and each $dp[i]$ depends on all $dp[j]$ for j less than i , so it needs a nested loop rather than a single pass.

3. Comparing the Recurrence Relations and Decision-Making

Aspect	Assembly Line Scheduling	Word Break
Nature of state	$f[i][j]$: minimum cost to reach a station	$dp[i]$: yes/no reachability of a prefix
Type of value stored	Numeric (minimum time)	Boolean (true/false)
Number of choices per state	Fixed - only 2 choices (stay or switch line)	Variable - up to i choices (every possible split point j)
Dependency pattern	Depends only on the previous column ($j-1$)	Depends on all previous states (0 to $i-1$)
Type of optimisation	Minimisation problem (shortest time)	Existence / decision problem (is it possible)
Combining sub-problems	Takes the min of two candidate paths	Takes the OR of many candidate splits

Even though both use DP, the decision-making style is different. Assembly line scheduling is a minimisation problem where, at every step, only two options exist (continue on the same line or switch lines), so the recurrence is a simple min of two terms. Word break is a decision (reachability) problem where at every position there can be many possible previous split points, so the recurrence checks many possibilities using an OR condition. This shows that DP is not just "one formula" - the shape of the recurrence changes based on what kind of question is being asked (best value vs is it possible).

4. Differences in Constraints

- Assembly line scheduling has a fixed, small branching factor - only 2 lines, so only 2 choices at each station, regardless of how large n (number of stations) becomes.
- Word break has a branching factor that grows with the position - at position i , there can be up to i different split points to check, so the number of choices increases as the string gets longer.
- Assembly line scheduling works with numeric costs (times) that must satisfy the triangle-type property of taking a minimum; there is no concept of "validity" of a choice, every station must be visited.
- Word break depends on an external dictionary - the validity of a substring ($s[j..i-1]$ being a real word) is a lookup constraint, not a numeric cost, and if the dictionary is large, checking membership efficiently (using a hash set) becomes important.
- Assembly line scheduling assumes the input (times) are always given and finite, so a solution always exists. Word break may have no valid segmentation at all, so the answer can genuinely be "not possible".

5. Complexity Evaluation

Problem	Time Complexity	Space Complexity
Assembly Line Scheduling	$O(n)$ - single pass over n stations, constant work per station	$O(n)$ for the table, or $O(1)$ with two variables
Word Break	$O(n^2)$ - for each of n positions, checking up to n split points; $O(n^2 * L)$ if substring hashing/comparison of length L is counted	$O(n)$ for the dp boolean array (plus dictionary storage, typically a hash set of size proportional to number of words)

Assembly line scheduling is linear time because the number of choices per station is fixed at 2, so the total work is proportional to n . Word break is quadratic (or more) because the number of choices per position grows with the position itself, and checking whether a substring belongs to the dictionary also costs time depending on how it is implemented (a hash set lookup is close to $O(1)$ on average, but very long substrings still cost time to hash or slice).

6. Scalability

Assembly line scheduling scales very well - since each station only looks at 2 choices, it stays efficient even for a very large number of stations (n in the order of millions), and can be solved with $O(1)$ extra space by only keeping the previous column's values instead of the full table.

Word break scales less comfortably because of its $O(n^2)$ nature. For long strings, this can become slow. In practice, this can be optimised a bit by limiting the split search to only go back as far as the length of the longest word in the dictionary (instead of checking all the way back to 0), which reduces the checks per position and helps scalability when the dictionary's word lengths are bounded.

7. Insights on DP Versatility

- DP is useful whenever a problem has optimal substructure (the best/valid answer for the whole problem can be built from answers to smaller parts) and overlapping sub-problems (the same smaller part is needed again and again).
- The same DP idea can be used for very different goals - minimising a cost (assembly line) or checking if something is possible (word break) - just by changing what is combined at each step: `min()` for optimisation, `OR()` for existence.
- The size and shape of the state space (how many choices per state, and how far back a state depends) directly decides the algorithm's time complexity, so understanding the branching pattern of a problem is the key step in analysing DP efficiency.
- Fixed, small branching problems like assembly line scheduling tend to give clean linear-time DP solutions, while problems where the number of choices grows with the input size, like word break, tend to give polynomial-time solutions that need more careful optimisation for scalability.

Overall, this comparison shows that dynamic programming is not tied to one type of problem - scheduling, string segmentation, and many other problems all fit the DP pattern once they are broken down correctly into states and a recurrence relation. The main skill is identifying the right state definition and the right way to combine sub-problem answers for the specific question being asked.